

Chapter 2: Problem Solving

4 hours | 7 marks

"A problem well stated is a problem half solved."

— Charles Kettering

The Hook — What Does It Mean to "Solve" Something?

Imagine you're lost in a maze.

You can:

1. **Wander randomly** — might work, but inefficient
2. **Follow the right wall** — a heuristic strategy
3. **Map the maze systematically** — guaranteed to work, might take time

How a *person* solves a maze and how an *AI* solves a maze are fundamentally different — but they share the same underlying structure.

This chapter is about **formalizing** what it means to have a problem, and how to represent it so a machine can reason about it.

2.1 Defining Problems as State Space Search

The Core Idea

Every problem in AI can be framed as: "I'm here. I want to be there. What sequence of actions gets me from here to there?"

This is called the **State Space** formulation.



The AI's job is to **find a path** through this space.

Formal Definition

A problem is defined by 5 components (memorize this — it's foundational):

Component	What It Is	Maze Example
Initial State	Where we start	Top-left cell
Actions	What we can do from any state	Move Up/Down/Left/Right
Transition Model	$\text{Result}(s, a) = s'$	Moving right from (1,1) $\rightarrow$ (1,2)
Goal Test	Are we done?	Are we at the exit?
Path Cost	How expensive is this path?	Number of steps taken

The solution is a sequence of actions that leads from the initial state to a goal state.

An **optimal solution** has the minimum path cost.

2.2 Problem Formulation

The Art of Abstraction

Here is a crucial insight: **Real-world problems are too messy. We must abstract.**

Example: Driving from Kathmandu to Pokhara

Real world includes:

- Road conditions, traffic lights, weather, fuel level, speed, other cars...

Abstracted state space:

- **State:** The city/town we're currently in
- **Actions:** Drive to an adjacent city
- **Goal:** Reach Pokhara
- **Path cost:** Distance in km

We threw away thousands of details, and the problem became solvable.

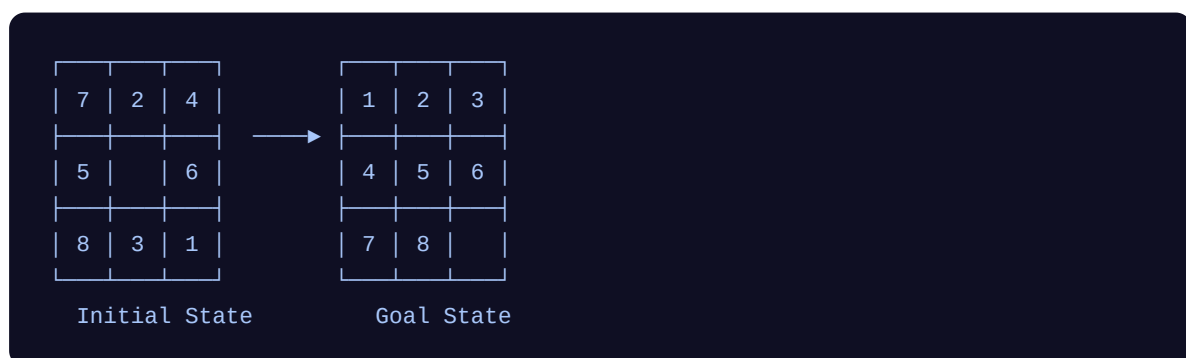
Key Insight: Good AI is about choosing the *right* abstraction — not too detailed (too complex) and not too coarse (loses important structure).

2.3 Problem Types

Type 1: Single-State Problems (Deterministic, Fully Observable)

The agent knows everything about the current state and what each action does.

Example: 8-puzzle (sliding tiles)



- States: All possible tile arrangements ($9!/2 \approx 181,440$ valid states)
- Actions: Slide blank Up/Down/Left/Right
- Goal: Reach the sorted arrangement

Type 2: Multiple-State Problems (Non-observable)

The agent doesn't know exactly which state it's in, so it tracks a *set* of possible states.

Example: A robot in a room with the lights off doesn't know its position. It must reason about *all* possible positions.

Type 3: Contingency Problems (Non-deterministic, partially observable)

Actions may have unexpected results. The agent must plan for multiple possibilities.

Example: Picking up an object — might slip. Must have a backup plan.

Type 4: Exploration Problems

The agent doesn't have a map at all. It must discover the state space as it goes.

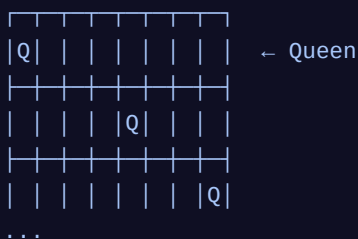
Example: A rover on Mars exploring unknown terrain.

2.4 Well-Defined Problems — The Classic Examples

These are the "textbook problems" AI researchers use to test algorithms. Know them deeply.

Problem 1: The 8-Queens Problem

Setup: Place 8 queens on an 8×8 chessboard so no two queens attack each other.



State: Board configuration

Initial State: Empty board

Actions: Place a queen in any non-attacked column

Goal Test: 8 queens placed, none attacking each other

Path Cost: Not relevant (we care about existence of solution, not cost)

Why it matters: The brute-force search space is $8^8 = 16.7$ million states. With good formulation (one queen per column), it drops to $8! = 40,320$. With constraints, even less.

Lesson: Formulation dramatically affects difficulty!

Problem 2: Missionaries and Cannibals

Setup: 3 missionaries and 3 cannibals must cross a river. The boat holds 2 people. At no point should cannibals outnumber missionaries on either bank.

State: (M_left, C_left, Boat_side)

e.g., (3, 3, left) = start state

Goal: (0, 0, right)

Constraint: On either bank, if missionaries > 0 , then missionaries $\geq$ cannibals.

Let's trace a few transitions:

```
State: (3,3,L)
↓ [1M, 1C cross →]
State: (2,2,R)
↓ [1M returns ←]
State: (3,2,L)
...
```

This seems simple but has subtle dead ends. It's a classic state-space graph problem.

Problem 3: Water Jug Problem

Setup: Two jugs — 4L and 3L. No markings. Fill exactly 2L.

State: (x, y) where x = water in 4L jug, y = water in 3L jug

Actions:

1. Fill 4L jug completely

2. Fill 3L jug completely
3. Empty 4L jug
4. Empty 3L jug
5. Pour from 4L to 3L (until one full or one empty)
6. Pour from 3L to 4L

Goal state: (2,) or (, 2)

Solution trace:

```
(0,0) → fill 4L → (4,0)
(4,0) → pour 4L into 3L → (1,3)
(1,3) → empty 3L → (1,0)
(1,0) → pour 1L into 3L → (0,1)
(0,1) → fill 4L → (4,1)
(4,1) → pour 4L into 3L → (2,3) ← GOAL: 4L jug has 2L!
```

Insight: What looks like a puzzle is really just graph traversal!

2.5 Constraint Satisfaction Problems (CSPs)

The Paradigm Shift

In standard search: find a **path** from start to goal.

In CSPs: find a **state** that satisfies all constraints.

Formal definition of a CSP:

- A set of **variables** $\{X_1, X_2, \dots, X_n\}$
- Each variable has a **domain** (possible values)
- A set of **constraints** (relationships between variables)

Goal: Assign values to all variables such that all constraints are satisfied.

Classic Example: Map Coloring

Problem: Color the map of Australia so no two adjacent regions share a color.



Variables: WA, NT, Q, SA, NSW, V, T

Domain: {Red, Green, Blue}

Constraints: Adjacent regions $\neq$ same color

A valid coloring:

WA = Red, NT = Green, Q = Red
 SA = Blue, NSW = Green, V = Red, T = Green

This is much more elegant than exhaustively searching all $3^7 = 2,187$ combinations — constraints prune the search space dramatically.

CSP in the Real World

Problem	Variables	Constraints
Exam scheduling	Exam time slots	No two exams a student takes overlap
Sudoku	Numbers in cells	No repeats in row/column/box
Circuit design	Component values	Electrical constraints
University timetabling	Class times/rooms	No room conflicts, no teacher conflicts

2.6 Game Playing — The AI Adversary

What Changes in Games?

Until now, we had one agent, one goal. In **games**:

- There are two (or more) players

- One player's gain is another's loss (**zero-sum**)
- The environment is adversarial — the opponent *actively tries to defeat you*

This changes everything about search.

Examples: Chess, tic-tac-toe, Go, Checkers

We define the game as:

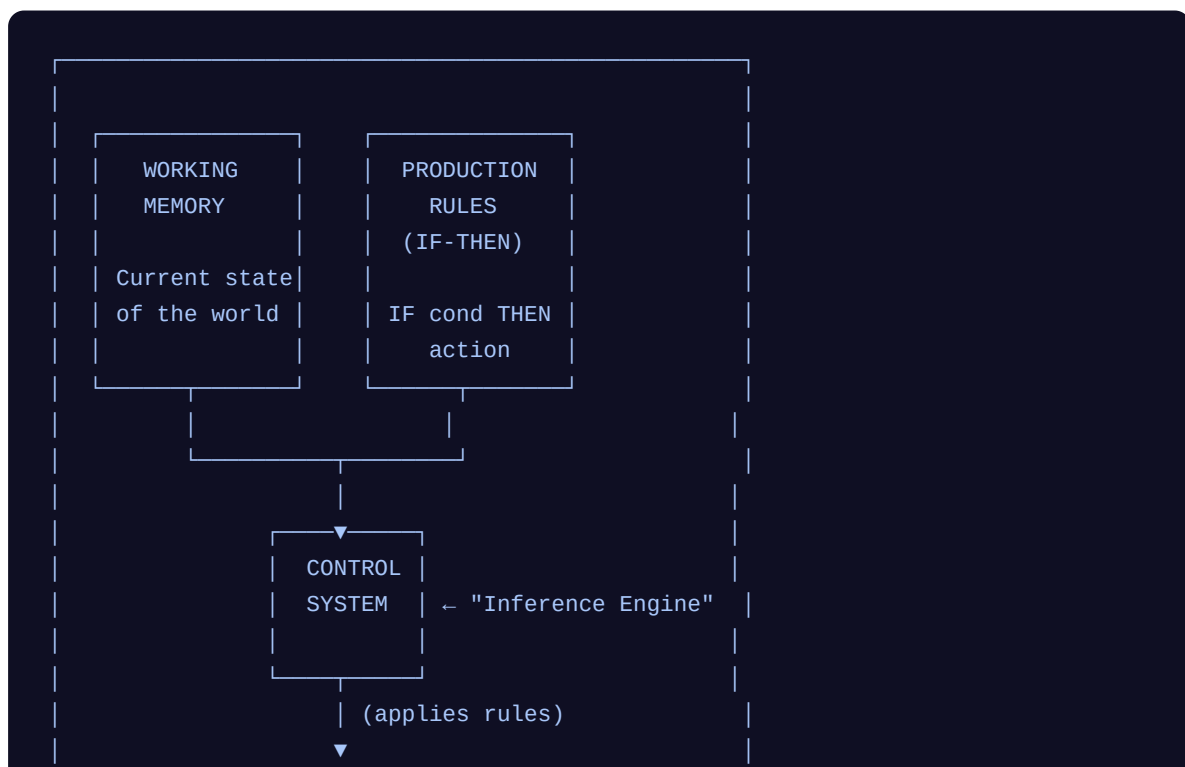
- **State:** Current board configuration
- **Players:** MAX (you, trying to maximize score) and MIN (opponent, minimizing your score)
- **Terminal Test:** Is the game over?
- **Utility Function:** What is the score at terminal states? (e.g., +1 for win, -1 for loss, 0 for draw)

The key algorithm for games is **Minimax** — but that's Chapter 3's territory.

2.7 Production Systems

What Is a Production System?

A **production system** is an AI architecture consisting of three parts:





Updates Working Memory

Components:

1. **Working Memory (WM):** Current beliefs about the world (facts)
2. **Production Rules:** "IF condition THEN action" rules
3. **Control System (Inference Engine):** Decides which rule to apply

Example: Medical Diagnosis Production System

```
Rule 1: IF (patient has fever) AND (patient has rash) THEN (suspect measles)
Rule 2: IF (patient has fever) AND (patient has sore throat) THEN (suspect strep)
Rule 3: IF (suspect measles) AND (patient not vaccinated) THEN (confirm measles risk)
```

Given working memory: {fever=true, rash=true, vaccinated=false}

Execution:

1. Rule 1 fires → adds "suspect measles" to WM
2. Rule 3 fires → adds "confirm measles risk" to WM

This is the basis of **Expert Systems** (Chapter 7)!

Math Intuition Corner

State Space Size

Understanding why search is hard requires understanding state space size:

8-puzzle: 9 tiles, 9 positions = $9! = 362,880$ states (manageable)

15-puzzle: $16! = 20,922,789,888,000$ states (need smart search!)

Chess: $\sim 10^{43}$ legal positions (impossible to exhaustively search!)

This is why **intelligent search** (Chapters 3) is needed — brute force doesn't scale.

Path Cost Formulation

```
g(n) = total cost of path from start to node n
      =  $\sum \text{step\_costs}(s, a, s')$  for all steps taken
```

For simple problems (equal cost steps): $g(n)$ = depth of node

For problems with varying costs: must track actual cumulative cost



Video Resources

Topic	Video	Why Watch
State Space Search intro	State Space Search — Computerphile	Clear UK academic explanation
8-puzzle visualization	Search 8 Puzzle visualization	Watch it being solved live
Constraint Satisfaction	CSPs explained	Conceptual walkthrough
Production Systems	Expert Systems overview	Context for Chapter 7



Worked Example: Formulate the Problem Formally

Task: A robot vacuum must clean all dirty cells in a 2x2 grid.

State: (robot_position, left_clean, right_clean, up_clean, down_clean)

Or more simply for 2 cells: (position, [dirty/clean, dirty/clean])

Grid:

```

┌───┬───┐
│ A │ B │
├───┴───┤
│dirty│dirty│
└───┬───┘
```

Robot starts at A

Initial State: (A, dirty, dirty)

Actions: Left, Right, Suck (clean current cell)

Transitions:

- (A, d, d) + Suck → (A, c, d)

- $(A, c, d) + \text{Right} \rightarrow (B, c, d)$
- $(B, c, d) + \text{Suck} \rightarrow (B, c, c) \leftarrow \text{GOAL!}$

Goal Test: Both cells clean

Optimal Path: Suck, Right, Suck (3 actions)

⚡ Exam-Ready Summary

The 5-Component Problem Definition

`Problem = {Initial State, Actions, Transition Model, Goal Test, Path Cost}`

Problem Types Quick Reference

Type	Observation	Determinism	Example
Single-state	Full	Deterministic	8-puzzle
Multiple-state	None	Deterministic	Robot in dark
Contingency	Partial	Non-deterministic	Slippery robot
Exploration	Full	Deterministic	Mars rover

CSP vs Standard Search

Aspect	Standard Search	CSP
Goal	Find a path	Find a valid assignment
Solution	Sequence of actions	Value assignment
Example	8-puzzle	Map coloring, Sudoku

Production System Components

1. **Working Memory** — current world state
2. **Production Rules** — IF-THEN rules

3. Control System — picks and applies rules

✓ Chapter 2 Checklist

Before moving to Chapter 3, ensure you can:

- ☐ Define all 5 components of a problem formally
 - ☐ Formulate the 8-Queens, Water Jug, and M&C problems formally
 - ☐ Explain the difference between all 4 problem types
 - ☐ Define CSP with variables, domains, and constraints
 - ☐ Give 3 real-world CSP examples
 - ☐ Draw and explain the 3 components of a production system
 - ☐ Explain why game-playing is different from single-agent search
-

🔗 Navigation

← **Previous:** [Chapter 1 — Introduction](#)

→ **Next:** [Chapter 3 — Search Techniques](#)

🏠 **Home:** [README](#)